

ダッシュボード画面の実装

ダッシュボード画面（ `DashboardView.vue` ）は、ユーザーがログインした後に最初に目にする「アプリのホーム画面（ハブ）」となる重要な画面です。

今回は仕様書に記載されている要件をベースに、Java（SpringBoot）とVue.js（フロントエンド）の両面から設計していきます。

今回構築するダッシュボード画面の仕様

仕様書に基づき、画面を「上段」「中段」「下段」の3つのブロックに分けて構成します。

- **上段（お知らせエリア）：**
管理者からのお知らせや、申請の承認・差戻し通知を表示するエリア
- **中段（メインアクション）：**
「打刻画面へ」遷移するボタンと「ログアウト」ボタン
- **下段（メニューリンク）：**
「予定申請」「打刻内容編集申請」「アカウント編集」など、各機能へ遷移するリンク・ボタン群

バックエンド（Java）の実装

まずは、ダッシュボードを開いたときにお知らせ情報を取得するためのAPIエンドポイントを準備します。仕様書にある「announcement」テーブルからデータを取得するイメージです。

1. Announcement.java (Entity) の作成

※すでに作成済みの場合はスキップしてください。

```
package com.example.myapp.entity;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.Data;

@Entity
@Table(name = "announcement")
@Data
```

```
public class Announcement {
    @Id
    private String announcementId; // UUID
    private String announcementTitle;
    private String announcementAbout;
    private Integer isDeletable; // 0:削除不可, 1:削除可能
}
```

2. AnnouncementRepository.java の作成

```
package com.example.myapp.repository;

import com.example.myapp.entity.Announcement;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface AnnouncementRepository extends JpaRepository<Announcement, String> {
    // 最新のお知らせをすべて取得（必要に応じて並び替えメソッドにしてもOK）
    List<Announcement> findAll();
}
```

3. DashboardController.java の作成

ダッシュボード用のお知らせデータをフロントエンドに返すコントローラーを新設します。

```
package com.example.myapp.controller;

import com.example.myapp.entity.Announcement;
import com.example.myapp.repository.AnnouncementRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/dashboard")
@CrossOrigin(origins = "*") // 環境に合わせて調整
public class DashboardController {
```

```
@Autowired
private AnnouncementRepository announcementRepository;

/**
 * ダッシュボード用のお知らせ一覧を取得する
 * GET /api/dashboard/announcements
 */
@GetMapping("/announcements")
public ResponseEntity<List<Announcement>> getAnnouncements() {
    List<Announcement> list = announcementRepository.findAll();
    return ResponseEntity.ok(list);
}
}
```

フロントエンド（Vue.js）の実装

続いて、Vue.js側に `DashboardView.vue`（または `Dashboard.vue`）を作成します。

画面遷移（ルーティング）には `Vue Router` を利用する想定で、ボタンを押したときに各画面へ飛べるように `router.push` を組み込みます。

`DashboardView.vue` の作成

```
<template>
  <div class="dashboard-container">
    <header class="dashboard-header">
      <h1>ダッシュボード</h1>
      <span class="user-welcome">ようこそ、{{ userName }} さん</span>
    </header>

    <section class="announcement-section">
      <h2>📢 お知らせ・通知</h2>
      <div v-if="announcements.length === 0" class="no-announcement">
        現在新しいお知らせはありません。
      </div>
      <div v-else class="announcement-list">
        <div v-for="item in announcements" :key="item.announcementId" class="announcement-card">
```

```
        <h3>{{ item.announcementTitle }}</h3>
        <p>{{ item.announcementAbout }}</p>
    </div>
</div>
</section>

<section class="main-actions">
    <button @click="navigateTo('/attendance')" class="btn-main btn-attendance">
        ⌚ 打刻画面へ移動
    </button>
    <button @click="handleLogout" class="btn-main btn-logout">
        🚪 ログアウト
    </button>
</section>

<section class="menu-section">
    <h2>🔧 各種メニュー</h2>
    <div class="menu-grid">
        <button @click="navigateTo('/schedule-demand')" class="btn-menu">
            📅 予定申請
        </button>
        <button @click="navigateTo('/timechange-demand')" class="btn-menu">
            📝 打刻内容編集申請
        </button>
        <button @click="navigateTo('/profile-edit')" class="btn-menu">
            👤 アカウンド情報編集
        </button>
    </div>
</section>
```



```

        <button v-if="isAuth === 1" @click="navigateTo('/admin')" class="btn-menu btn-admin">
            🏰 【管理者】各種管理画面
        </button>
    </div>
</section>
</div>
</template>

<script setup>
import { ref, onMounted } from 'vue';
import { useRouter } from 'vue-router';
import apiClient from '../api/axios'; // 既存の共通APIクライアント

// 親コンポーネントやログイン状態から受け取るプロパティ（仮受け）
const props = defineProps({
  accountId: { type: String, default: '' },
  userName: { type: String, default: 'サンプルユーザー' },
  isAuth: { type: Number, default: 0 } // 0:一般, 1:管理者
});

const router = useRouter();
const announcements = ref([]);

// お知らせデータの取得
const fetchAnnouncements = async () => {
  try {
    const response = await apiClient.get('/dashboard/announcements');
    announcements.value = response.data;
  } catch (error) {
    console.error('お知らせの取得に失敗しました', error);
  }
};

```

```
// 画面遷移ハンドラー
const navigateTo = (path) => {
  router.push(path);
};

// ログアウト処理
const handleLogout = () => {
  if (confirm('ログアウトしますか?')) {
    // セッションやトークンのクリア処理をここに記述
    localStorage.removeItem('token'); // 例
    router.push('/login');
  }
};

onMounted(() => {
  fetchAnnouncements();
});
</script>
```

```
<style scoped>
.dashboard-container {
  max-width: 800px;
  margin: 0 auto;
  padding: 200px 20px 20px; /* ヘッダー等との兼ね合い */
  font-family: sans-serif;
}

.dashboard-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 2px solid #34495e;
  padding-bottom: 10px;
  margin-bottom: 20px;
}

/* 上段: お知らせスタイル */
.announcement-section {
  background-color: #f8f9fa;
  padding: 15px;
  border-radius: 8px;
  border-left: 5px solid #3498db;
  margin-bottom: 25px;
}

.announcement-card {
  background: white;
  padding: 10px 15px;
  margin-top: 10px;
  border-radius: 4px;
  box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}
```

```
/* 中段: メインアクション */
.main-actions {
  display: flex;
  gap: 15px;
  margin-bottom: 30px;
}
.btn-main {
  flex: 1;
  padding: 15px;
  font-size: 1.2rem;
  font-weight: bold;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.2s;
}
.btn-attendance { background-color: #2ecc71; color: white; }
.btn-attendance:hover { background-color: #27ae60; }
.btn-logout { background-color: #95a5a6; color: white; }
.btn-logout:hover { background-color: #7f8c8d; }

/* 下段: メニューグリッド */
.menu-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 15px;
  margin-top: 15px;
}
.btn-menu {
  padding: 20px;
  font-size: 1rem;
  background-color: #34495e;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.2s, transform 0.1s;
}
.btn-menu:hover { background-color: #2c3e50; transform: translateY(-2px); }
.btn-admin { background-color: #e67e22; }
.btn-admin:hover { background-color: #d35400; }
</style>
```

SecurityConfig.java への許可追加

新しく `/api/dashboard/` というエンドポイントを作成したため、Spring SecurityにこのURLを通すための設定を加えます。

```
// SecurityConfig.java の一部
.requestMatchers(
    "/api/users/register",
    "/api/users/login",
    "/api/attendance/history/**",
    "/api/attendance/punch",
    "/api/dashboard/**" // ← これを追加！
).permitAll()
```

次のステップ：Vue Routerの設定（URLの登録）

ログイン後に「ダッシュボード」と「打刻画面」を自由に行き来できるようにするためには、当初の仕様書にある通り、画面上部のヘッダーエリアに「ハンバーガーメニュー（メニューボタン）」を実装するのが最適です。

これを実現するために必要なステップは以下の2点です。

1. **Vue Router**（ルーティング設定）への登録（画面のURLとコンポーネントの紐付け）
2. **共通ヘッダーコンポーネント、または各画面へのハンバーガーメニューの実装**

具体的な手順とコード例を解説します。

Vue Router の設定（URLの登録）

まずは、ブラウザのURL（`/dashboard` や `/attendance`）と、作成したコンポーネントが正しく結びつくようにルーターを設定します。

ルーターの設定ファイル（通常は `src/router/index.js` または `src/router.js`）を開き、以下のように `DashboardView.vue` をルーティングに追加してください。

```
// router/index.js のイメージ
import { createRouter, createWebHistory } from 'vue-router';
import LoginView from '../views/LoginView.vue';
import AttendanceBoard from '../views/AttendanceBoard.vue';
import DashboardView from '../views/DashboardView.vue'; // ← 追加

const routes = [
  { path: '/login', component: LoginView },
  { path: '/attendance', component: AttendanceBoard },
  { path: '/dashboard', component: DashboardView }, // ← 追加
  { path: '/', redirect: '/login' } // 初期アクセスはログインへ
];
```

```
const router = createRouter({  
  history: createWebHistory(),  
  routes  
});
```

// ※ここにログイン状態をチェックするナビゲーションガード（仕様書要件）を後ほど追加できます

```
export default router;
```

共通ヘッダー（ハンバーガーメニュー）の実装

各画面（打刻画面、ダッシュボード画面、その他今後増える申請画面）で共通して使えるヘッダーコンポーネント（`AppHeader.vue`）を新設するのが、最もスマートで管理しやすい方法です。

1. `components/AppHeader.vue` を新設する

メニューボタン（漢数字の「三」のようなアイコン）をクリックすると、メニューがパッと開き、各画面へのリンクが表示される仕組みを作ります。


```
<template>
  <header class="app-header">
    <div class="header-title" @click="router.push('/dashboard')">
      入退室管理システム
    </div>

    <button class="hamburger-btn" @click="toggleMenu" :class="{ 'is-open': isMenuOpen }">
      <span class="bar"></span>
      <span class="bar"></span>
      <span class="bar"></span>
    </button>

    <transition name="slide">
      <nav v-if="isMenuOpen" class="dropdown-menu">
        <ul>
          <li><a @click="navigate('/dashboard')">ダッシュボード</a></li>
          <li><a @click="navigate('/attendance')">打刻画面</a></li>
          <li><a @click="navigate('/schedule-demand')">予定申請</a></li>
          <li><a @click="navigate('/timechange-demand')">打刻内容編集申請</a></li>
          <li class="menu-divider"></li>
          <li><a @click="handleLogout" class="logout-link">ログアウト</a></li>
        </ul>
      </nav>
    </transition>
  </header>
</template>
```

```
<script setup>
import { ref } from 'vue';
import { useRouter } from 'vue-router';

const router = useRouter();
const isMenuOpen = ref(false);

const toggleMenu = () => {
  isMenuOpen.value = !isMenuOpen.value;
};

const navigate = (path) => {
  isMenuOpen.value = false; // メニューを閉じる
  router.push(path);
};

const handleLogout = () => {
  isMenuOpen.value = false;
  if (confirm('ログアウトしますか?')) {
    localStorage.removeItem('token'); // 必要に応じて
    router.push('/login');
  }
};
</script>
```

```
<style scoped>
.app-header {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 60px;
  background-color: #2c3e50;
  color: white;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 20px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.2);
  z-index: 1000;
  box-sizing: border-box;
}

.header-title {
  font-size: 1.2rem;
  font-weight: bold;
  cursor: pointer;
}
```

```
/* ハンバーガーボタンのスタイル */  
.hamburger-btn {  
  background: none;  
  border: none;  
  cursor: pointer;  
  display: flex;  
  flex-direction: column;  
  justify-content: space-between;  
  width: 30px;  
  height: 20px;  
  padding: 0;  
}  
  
.bar {  
  display: block;  
  width: 100%;  
  height: 3px;  
  background-color: white;  
  border-radius: 2px;  
  transition: all 0.3s ease;  
}
```

```
/* メニューが開いている時の三本線の変形 (x印にする演出) */
.hamburger-btn.is-open .bar:nth-child(1) {
  transform: translateY(8px) rotate(45deg);
}
.hamburger-btn.is-open .bar:nth-child(2) {
  opacity: 0;
}
.hamburger-btn.is-open .bar:nth-child(3) {
  transform: translateY(-9px) rotate(-45deg);
}

/* ドロップダウンメニューのスタイル */
.dropdown-menu {
  position: absolute;
  top: 60px;
  right: 0;
  width: 250px;
  background-color: #34495e;
  box-shadow: -2px 4px 10px rgba(0,0,0,0.3);
  border-bottom-left-radius: 8px;
}

.dropdown-menu ul {
  list-style: none;
  margin: 0;
  padding: 10px 0;
}
```

```
.dropdown-menu li a {
  display: block;
  padding: 15px 20px;
  color: #ecf0f1;
  text-decoration: none;
  cursor: pointer;
  transition: background 0.2s;
}

.dropdown-menu li a:hover {
  background-color: #465c71;
}

.menu-divider {
  height: 1px;
  background-color: #2c3e50;
  margin: 5px 0;
}

.logout-link {
  color: #e74c3c !important;
}

/* アニメーション効果 */
.slide-enter-active, .slide-leave-active {
  transition: all 0.3s ease;
}
.slide-enter-from, .slide-leave-to {
  transform: translateY(-10px);
  opacity: 0;
}
</style>
```

2. 全体（App.vue）にヘッダーを組み込む

ログイン画面以外では常にこのヘッダーが表示されるように、メインの App.vue に配置します。現在のルーティング状況（route.path）を見て、ログイン画面や登録画面以外でヘッダーを表示する制御を入れます。

```
<template>
  <div id="app">
    <AppHeader v-if="showHeader" />

    <main class="main-content" :class="{ 'has-header': showHeader }">
      <router-view />
    </main>
  </div>
</template>
```

```
<script setup>
import { computed } from 'vue';
import { useRoute } from 'vue-router';
import AppHeader from './components/AppHeader.vue';

const route = useRoute();

// ログイン画面とユーザー登録画面ではヘッダーを隠す
const showHeader = computed(() => {
  return route.path !== '/login' && route.path !== '/register';
});
</script>

<style>
/* ヘッダー固定に伴い、コンテンツが下に隠れないように余白を作る */
.main-content.has-header {
  padding-top: 80px;
}
</style>
```


次に進めるステップ

1. `Vue Router` の設定ファイル に、ダッシュボード画面のパス（`/dashboard`）を追加してください。
2. `AppHeader.vue` を作成し、`App.vue` で読み込ませてみてください。

これが開通すると、画面右上の「ハンバーガーメニュー」をポチッと押すだけで、いつでも「ダッシュボード」と「打刻画面」をアニメーション付きで行き来できるようになります！

まずはルーターの登録あたりから進めてみましょう。記述中に不明な点があればいつでも声をかけてくださいね！